

Boole-ANN: Joint Physical Design for Structured Filtered Vector Search

Qionghao Huang
qhhuang@zjnu.edu.cn
Zhejiang Normal University
Jinhua, China

Feiyang Shu
feiyangshu@zjnu.edu.cn
Zhejiang Normal University
Jinhua, China

Changqin Huang
cqhuang@zju.edu.cn
Zhejiang University
Hangzhou, China

ABSTRACT

Filtered approximate nearest neighbor search retrieves vectors that are similar to a query while enforcing structured predicates over their attributes. It is increasingly used in retrieval, recommendation, and semantic search, where a relevant result is useful only if it also satisfies application constraints. Existing systems, however, optimize either a collection of filtered indexes, the execution of complex predicates, or a generic fallback, leaving reusable predicate fragments, their physical layouts, and their query plans to be designed separately.

We introduce *Boole-ANN*, a physical designer that makes these decisions jointly. *Boole-ANN* learns a shared representation of exact predicate supports from a representative workload and uses it to capture equality atoms, recurring conjunctions, and ordered blocks. For every reusable fragment, it considers alternative physical layouts and admits only the combinations that are useful under the available resource budget. Ordered fragments can be composed to answer previously unseen ranges and supported Boolean expressions without materializing every query endpoint. At runtime, *Boole-ANN* selects a recall eligible plan, validates the predicate, and invokes a complete exact fallback within this grammar whenever no specialized plan is appropriate.

We establish the correctness of the support algebra and the exact execution tier. Experiments on real and controlled workloads show that *Boole-ANN* outperforms evaluated released state-of-the-art methods at recall-qualified endpoints across equality, conjunction, and range workloads, while preserving predicate validity under explicit resource and tail-coverage goals.

KEYWORDS

filtered approximate nearest neighbor search, vector databases, physical design, graph indexes, Boolean predicates, query optimization

PVLDB Reference Format:

Qionghao Huang, Feiyang Shu, and Changqin Huang. Boole-ANN. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

The source code, data, and/or other artifacts have been made available at <https://github.com/qhhuang-ai/booleann>.

1 INTRODUCTION

Structured filtered ANN. Approximate nearest-neighbor (ANN) search returns database vectors close to a query; structured filtering restricts them by Boolean or numeric metadata. The result must obey the predicate and approximate the nearest valid top- k . We support equality, two-atom conjunction, one-dimensional closed range and complement, and two-clause range DNF; a clause may be equality-guarded, and the clauses may reference different attributes.

Predicates as physical design. Workloads repeatedly ask related predicates whose valid-record sets (*supports*) share atoms and subexpressions but differ in cardinality. The same support can be scanned as packed vectors, intersected through postings or bitmaps, or searched with a local graph; its best realization depends on size, reuse, and recall target.

Physical-design tensions. Three tensions define the design problem. *Reuse versus coverage* calls for complete supports that can serve many formulas without materializing every query predicate. *Representation versus execution* arises because the same support may favor postings, bitmaps, packed vectors, or a local graph as its cardinality and reuse change. *Specialization versus safe service* requires these choices to share explicit storage, build, serving-memory, and calibrated fast-plan coverage goals while predicate checks and a complete exact fallback remain available for every supported formula.

Design landscape. Label-aware graphs serve unpredictable predicates but filter during navigation, whereas predicate-specific graphs remove that work only for the supports they cover. Index collections, complex-filter planners, and ordered indexes address complementary parts of this tradeoff [6, 10, 20, 21, 25]. This leads to one question: *can a budgeted design share exact fragments, choose their layouts and recall-eligible plans, and compose predicates not materialized in full?*

Core contribution: Boole-ANN. *Boole-ANN* treats a reusable support, its layout, and its executable plan as one design object. Equality atoms and endpoint-independent ordered blocks are leaves of an exact-support DAG whose intersection, union, and difference nodes compose unseen conjunctions and range endpoints without caching query answers. An offline advisor chooses postings, bitmaps, packed vectors, or local graphs under storage, build, serving-memory, and tail-coverage constraints. At runtime, a manifest covers each predicate, authorizes an exact or recall-eligible plan,

checks the original predicate, and reranks exact distances; within the supported grammar, an exact fallback handles every unsupported specialization. Layout savings remain in the shared budget and can fund other fragments and plans. A common catalogue and manifest apply this contract across categorical, packed-prefix, and ordered execution backends, giving their specialized assets one design vocabulary and accounting model.

Additional contributions.

- An exact support algebra with validity proofs; a disjoint block family composes any one-dimensional interval using N stored memberships in total.
- A workload- and fast-plan-coverage-aware advisor for joint fragment–layout–plan selection, and a validated manifest executor with exact fallback.
- Protocol-matched, recall-qualified comparisons with released baselines from 10^5 to 10^8 vectors. At 100M, one post-freeze point has higher observed recall than Parlay-IVF’s highest-recall point while delivering $14.985\times$ the QPS of its fastest point; Parlay-IVF occupies $3.900\times$ as much persistent state. Controlled interventions isolate materialization, layout, and routing effects.

2 RELATED WORK

Workload-driven physical design. Classical advisors choose indexes from a workload, cost model, and storage constraint [5]; joint view/index selection adds reusable materialized results [1]. Boole-ANN brings this separation to structured vector search: supports are logical candidates, while joins, scans, and traversals are physical choices.

Global label-aware graphs. Filtered-DiskANN [7], ACORN [14], UNG [4], and JAG [23] modify graph construction or traversal so one structure can serve many predicates. NaviX [16], RWalks [2], FAVOR [18], MCI [26], and Tag-Filtered ANN [13] further develop predicate-agnostic, diffused, selectivity-aware, compressed, or hybrid graphs. This lane favors an unpredictable vocabulary but retains filtering within navigation. Boole-ANN instead spends a declared budget on recurring exact supports and retains the generic complete fallback within the supported grammar for the remainder.

Index collections and complex-filter planning. SIEVE [10] is the closest offline-design prior: it builds an HNSW subindex collection from workload history, tunes graph density under a memory/recall constraint, and chooses a subsuming subindex or brute force; its set-cover extension unions graph subindexes for disjunctions. VecFlow selects GPU layouts for one filter at a time [22]. PathFinder plans over DNF using attribute-specific proximity graphs [21]; Compass feeds a shared candidate queue from vector and relational indexes [25]; Parlay-IVF combines cluster and posting state [17]. Boole-ANN couples cross-predicate support admission with alternative layouts and checked composition; Sections 6.3 and 6.4 isolate that coupling.

Ordered predicates. The B-Window Search Tree (β -WST) [6], SeRF [29], iRangeGraph [24], WoW [20], and RNSG [28] specialize

navigation for numeric intervals. β -WST recursively partitions label order and composes window indexes; Boole-ANN shares fixed blocks with Boolean supports, assigns alternative exact/graph layouts, and selects checked plans. Mesh is workload- and memory-aware for spatial ranges [19]; DSG and DIGRA support dynamic scalar ranges [9, 15]; KHI, Garfield, and MSTG extend the space to multi-attribute boxes, GPUs, and interval-valued objects [11, 12, 27]. We compare released scalar-range implementations directly and use same-kernel controls for mixed-DNF composition.

System integration. Recent system-level evidence on FAISS, Milvus, and pgvector shows that hybrid exact/approximate execution and optimizer choices can dominate the underlying ANN index [3]. PipeANN-Filter explores a verified superset through an SSD-oriented speculative-filtering design [8]. Boole-ANN targets physical design and execution for a page-resident collection.

Table 1: Physical-design units and optimization scope in the closest lines of work.

Line	Representative systems	Physical unit and optimized decisions
shared / partitioned graph index collection / planner	Parlay-IVF, ACORN, UNG, JAG, SIEVE, VecFlow, PathFinder, Compass	one graph or cluster–posting state; filter-aware traversal; selected subgraphs or attribute indexes; parameter, layout, or online-plan choice
ordered / specialized	β -WST, SeRF, iRangeGraph, Mesh, WoW	windows, range graphs, or spatial cells specialized to an ordered predicate class
Boole-ANN	this work	compositional exact supports; joint fragment, layout, and authorized-plan selection

Table 1 locates the design boundary. Boole-ANN’s distinguishing physical-design unit is a compositional exact support whose admission, layout, and authorized plan share one budget. Its support algebra and checked execution make that unit reusable across Boolean and ordered formulas.

3 PHYSICAL-DESIGN MODEL

Let $D = \{(x_i, a_i)\}_{i=1}^N$ contain vectors $x_i \in \mathbb{R}^d$ and structured attributes a_i , and let $\delta(q, x_i)$ be the dataset’s distance. Predicate φ defines support $X_\varphi = \{i : \varphi(a_i) = 1\}$. With $k_\varphi = \min(k, |X_\varphi|)$, the reference answer $T_k(q, \varphi)$ is the first k_φ IDs in the stable $(\delta(q, x_i), i)$ order on X_φ ; well-formed output contains exactly k_φ distinct predicate-valid IDs. strict recall of output $\hat{T}$ is $|\hat{T} \cap T_k|/k_\varphi$ when $k_\varphi > 0$ (and one for a correctly empty answer). Tie-aware recall accepts any predicate-valid representative at the k_φ th-distance cutoff. We call a result *exact* when this tie-aware recall is one and every reported distance is exact; *stable exact* additionally requires the serialized ID set T_k .

The design vocabulary follows one chain: predicate φ defines an ID set X_φ ; a *fragment* is a reusable logical node for such a set; an *asset* realizes one fragment in a physical layout; a *plan* combines assets with operators; and a *manifest* is the budgeted set of assets and plans authorized for execution.

A logical *fragment* f denotes a reusable exact support P_f . Boole-ANN represents fragments as a hash-consed DAG:

- atom f_a with $P_{f_a} = X_a$;
- fixed block $f_{g,j}$ with $P_{f_{g,j}} = \{i : A(a_i) \in I_{g,j}\}$ for a candidate disjoint partition family g ;

- internal node $f = u \cap v$, $u \cup v$, or $u \setminus v$, whose support is the corresponding exact set operation on P_u and P_v .

Syntactically equal subexpressions share one node. The miner extracts subexpressions from calibration predicates, adds base-derived atom and block leaves, and rejects candidates below a minimum reuse or above a declared expression-size cap. The candidate set is fixed before evaluation; an uncovered expression is compiled from available leaves. A recurring pair is simply the node $f_a \cap f_b$, not a special-case index type.

From fragments to executable plans. A fragment is logical state, not yet an index. A physical asset $j = (f, \ell)$ couples it to a layout, and an executable candidate (j, o_θ) adds a compatible operator and frozen parameters. A plan may consume several assets, but every plan ends with deduplication, an exact predicate check, and a global exact-distance top- k merge. Table 2 makes the separation concrete. For example, $a \wedge b$ can use a materialized pair graph, filter a primary posting through a secondary bitmap, or merge two sorted postings; those are three physical plans for one logical predicate, not three unrelated systems. Joint selection chooses fragment admission, physical layout, and executable plan in one budgeted optimization, allowing savings from one decision to be reinvested in the others. In the strict two-stage control, fragment admission is frozen first; layout encoding and plan assignment can use only those fragments and cannot refill capacity released by a later layout choice. Where we compare the two, they receive the same candidate catalogue, calibration data, byte cap, execution kernels, and exact fallback. Decision coupling is the only changed factor.

Table 2: Representative fragment–layout–operator plans. Every plan checks the original predicate on global IDs before exact-distance merging.

Predicate	Assets and operator	Recall semantics
a	atom pack + scan	exact
a	atom graph + beam	empirical
$a \wedge b$	intersection-node pack/graph	exact / empirical
$a \wedge b$	posting + bitmap semi-join	exact
$a \wedge b$	two postings + merge	exact
$[l, r]$	overlapping blocks + scan/beam	exact / empirical
$t \wedge [l, r]$	ordered token owner + checked prefix scan	exact
DNF	shared DAG cover + checked union	exact / plan-level empirical

A physical asset $j = (f, \ell)$ uses a layout $\ell \in \{\text{posting, bitmap, packed, graph}\}$. A stored-ID asset is complete only when its checked local/global map is a bijection onto the fragment’s declared exact support; this *completeness invariant* is independent of ANN recall. Candidate j has a measured resource vector $\mathbf{c}_j = (\text{persistent bytes, build seconds, serving bytes})$.

For each workload class q , the enumerator produces plans p . A plan names its required optional assets $R(p)$ and its operator sequence; every class retains an always-available exact fallback plan. The calibration harness measures latency and freezes a conservative planning cost L_{qp} . Approximate candidates are assigned before evaluation to an admission group $a(q, p)$ (predicate family, operator, and the declared support stratum when available); each exact plan is an always-eligible singleton group. An approximate group is eligible only when the lower bound for the complete plan—including union, deduplication, predicate checking, and global reranking—meets target ρ_a on calibration. This is a distributional guard over formulas

in the group. Let the fixed indicator $e_a \in \{0, 1\}$ equal one for exact singleton groups and for approximate groups that pass this guard. Binary x_j selects a physical candidate and z_{qp} selects one eligible plan. Let $s_{qp} = 1$ when the plan’s conservative latency estimate falls below planning threshold τ , and let $\eta \in [0, 1]$ be the required workload-mass coverage. With nonnegative workload frequencies w_q and resource budgets $\mathbf{B}$, let $\mathcal{J}_0$ contain assets not coupled to a block family. The designer solves

$$\begin{aligned}
& \min_{x, z} \sum_q w_q \sum_{p \in \mathcal{P}_q} L_{qp} z_{qp} \\
& \text{s.t.} \quad \sum_j \mathbf{c}_{jr} x_j \leq \mathbf{B}_r \quad \forall r, \quad \sum_{p \in \mathcal{P}_q} z_{qp} = 1 \quad \forall q, \\
& \quad z_{qp} \leq x_j \quad \forall q, p, j \in R(p), \\
& \quad x_j \leq \sum_{q, p: j \in R(p)} z_{qp} \quad \forall j \in \mathcal{J}_0, \\
& \quad z_{qp} \leq e_{a(q, p)} \quad \forall q, p, \\
& \quad \sum_q w_q \sum_p s_{qp} z_{qp} \geq \eta \sum_q w_q, \quad x_j, z_{qp} \in \{0, 1\}.
\end{aligned} \tag{1}$$

Here branch eligibility is insufficient: an approximate group passes only when its complete composed plan passes the frozen guard. Fallback plans have $R(p) = \emptyset$ and are always eligible, so every supported query remains executable and predicate-valid with a tie-aware-exact plan available; selecting an approximate family instead enforces the empirical group-calibration target. When the tail-coverage constraint is infeasible at a tight budget, the advisor reports the attainable coverage together with the resulting objective. Solver status is reported as optimal at zero gap; otherwise we report the incumbent, bound, and relative gap. In the degree-one layout-alternative topology (each asset has one assignment edge and each query chooses one fragment’s layout), the substitution $x_j = z_{qp}$ reduces Eq. (1) to a multiple-choice knapsack over the active budget dimensions, plus the tail-coverage side constraint when it is active; with one byte budget and no tail constraint this is the classical single-budget multiple-choice knapsack, and Lagrangian reduced-cost pruning plus a zero-gap residual core certifies the original optimum. Plans coupling multiple fragments remain in the general MILP. On query-free degree-one catalogues the reduction matches the original zero-gap objective and selected bytes through 10K candidates; Section 6.3 evaluates the degree-one 100K endpoint and a connected catalogue whose accelerated plans share two or three assets. Supports and exact byte costs come from base data; workload weights, latency and recall eligibility come only from the disjoint calibration split. Candidate block granularities compete with atom/pair layouts in the same budgets. A group variable selects all members of a candidate block partition together, and at most one partition family may be selected per numeric attribute. Its disjoint supports satisfy $\sum_j |P_{f_j}| = N$ exactly. Everything selected by Eq. (1), including online operator parameters, is frozen before held-out execution.

Let $A(g)$ denote the numeric attribute partitioned by candidate block family g . For each such family, let $G_g = \{(f_{g,j}, \ell_g) : j =$

$1, \dots, B_g\}$ and use a binary h_g to couple the whole partition:

$$\begin{aligned}
x_j &= h_g & \forall j \in G_g, \forall g, \\
h_g &\leq \sum_{q,p: R(p) \cap G_g \neq \emptyset} z_{qp} & \forall g, \\
\sum_{g: A(g)=A} h_g &\leq 1 & \forall \text{ numeric attributes } A, \\
h_g &\in \{0, 1\} & \forall g.
\end{aligned} \tag{2}$$

Thus selecting $h_g = 1$ charges every member of G_g in every applicable budget even if calibration touches only part of the domain. Families on different attributes may be selected together, which supports cross-attribute DNF; competing granularities for the same attribute remain mutually exclusive. Resource coefficients are incremental over mandatory base postings and counted once when an asset supports several plans. The design manifest records selected assets, enabled plan templates, operator parameters, cost coefficients, and fallback rules. Online execution may inspect predicate structure and immutable base support counts, but may not create a new pair, bitmap, block boundary, or beam setting.

PROPOSITION 1 (VALIDITY AND EXACT TIER). *Suppose every selected unit satisfies the completeness invariant and the executor evaluates φ exactly on every candidate before output. The units searched may be subsets or supersets of X_φ ; the executor returns only predicate-valid IDs. If exact enumeration of the selected units produces a candidate set C with $X_\varphi \subseteq C$, exact distance ranking of $\{i \in C : \varphi(a_i) = 1\}$ returns a distance-exact filtered top- k ; reference-stable tie breaking additionally returns T_k . Replacing enumeration by graph traversal preserves validity but makes ranked recall empirical.*

PROOF. Completeness makes every local-to-global ID a member of its declared support; the final exact predicate test removes every ID outside X_φ , including IDs introduced by a superset atom or a boundary block. Thus output validity does not require a unit itself to be contained in X_φ . If $X_\varphi \subseteq C$, filtering C by φ yields exactly X_φ ; exact global top- k ranking is therefore sufficient. Graph traversal may omit valid members of C , but the final predicate test still precludes invalid output. $\square$

PROPOSITION 2 (EXACT SCREENING BY A CERTIFIED LOWER BOUND). *Let C be the complete predicate support after any leave-one-out exclusion, and let $\underline{\delta}(q, x) \leq \delta(q, x)$ for every $x \in C$ with respect to the same exact scorer. Let $S \subseteq C$ be a fully scored seed with $|S| \geq k$, and let t_k be its k th distance. Discarding only rows with $\underline{\delta}(q, x) > t_k$ preserves a distance-exact top- k of C and, with reference-stable tie breaking, T_k . A failed bound, nonfinite value, insufficient shortlist, or cutoff tie invokes full exact enumeration of C .*

PROOF. The global k th distance in C is at most t_k . A discarded row has exact distance strictly greater than t_k and therefore cannot enter the top- k ; the strict test retains every cutoff tie. Full enumeration is exact by Proposition 1. $\square$

The online optimizer evaluates only plans enabled by the selected design. It estimates T_{join} from posting/bitmap reads, $T_{\text{scan}} \propto |P|d$, and T_{graph} from support, beam, and requested k , then chooses the least-cost recall-eligible plan. Estimates use observable base cardinalities, never query outcomes; all coefficients and beams are

calibrated once and frozen. The evaluation reports calibration/test separation, resource coverage, and forced materialization controls.

Calibration and conservative eligibility. We stratify calibration observations by predicate form, support decile, and operator. For joins, features are the two posting lengths, intersection estimate, bitmap presence, and packed-vector availability; scan features are candidate count and dimension; graph features are support, requested k , and beam. Robust log-linear fits predict the median, while an empirical upper-residual correction supplies the planning cost used by the designer. For an approximate plan, recall eligibility is a separate decision: its complete end-to-end plan’s one-sided lower confidence bound must exceed the requested target. Latency and recall are independent gates, so a low estimated cost does not establish eligibility. Exact plans are eligible by construction.

For SIFT100M, eligibility uses 200 unique calibration queries from eight predeclared pair classes (25 each). For each of the materialized pair-graph and pair-disabled fallback families, a one-sided 95% Student- t lower bound across the eight class-mean strict recalls must reach .99. The pooled family is the testing unit, and every fragment inherits one of these two predeclared empirical calibration guards. Admission, beams, and plan parameters are fixed before the 58 held-out pair classes are evaluated. This screen governs group-level empirical ranked recall rather than arbitrary-formula population coverage; support completeness, final predicate checking, and exact-fallback ranking retain the deterministic contract of Proposition 1. We report held-out per-class recall separately as a transfer diagnostic.

For the coverage-first study in Section 6.3, we first maximize the left-hand side of the coverage constraint under the same budgets and set η to that attainable optimum. Among coverage-optimal designs, calibration CVaR₉₅ and then the mean objective in Eq. (1) break ties. In deployments where η is an operator-specified coverage target, only the latter stages are needed. The mean stage can equivalently maximize a conservative saving $\underline{\Delta}_{qp}$ relative to the mandatory fallback. Thus η measures the workload mass assigned to tail-eligible plans; observed request p95 is measured separately. A candidate with an uncertain or non-positive lower-bound saving has no incentive to consume space. This coverage–tail–mean ordering is important: optimizing additive mean saving can leave slightly more than 5% of workload mass on a slow exact fallback and therefore miss the p95 target even when two additional fragments would cross the coverage threshold. Support and byte features come from the base, whereas frequencies and fitted performance come only from calibration. Development chooses the global budget and recall targets; named query-row and predicate-class test partitions never alter design, tuning, or plan eligibility.

4 SYSTEM

Figure 1 makes the new design object concrete. The offline designer may inspect base attributes and a declared calibration workload, but not development/final query vectors, ground truth, or final outcomes. It freezes a shared support DAG, selected layouts, and operator parameters. At serving time an unseen formula is covered by those frozen nodes; the executor may warm pages but cannot add an intersection or move an ordered breakpoint, and this transient working set is included in peak RSS. We call the evaluated

equality/conjunction implementation the *Boolean Conjunction Index* (BCI). The selected contiguous *Predicate-Cube* backend, denoted X3 in the evaluation, and the ordered-range backend use the same materialization-unit and checked-merge interface. They are executor backends of one design contract: both consume manifest rows that name complete supports, charged layouts, authorized plan templates, and exact fallbacks; only the predicate-specific compiled operators differ.

4.1 Offline units and admission

The designer hash-conses atom and fixed-block leaves plus recurring calibration subexpressions, computes their exact supports by posting algebra, then instantiates each eligible support–layout alternative. A candidate stores a complete local-to-global ID map; a cardinality assertion rejects partial or mixed builds. Short supports may receive contiguous vectors for exact SIMD scanning, high-support membership tests may receive bitmaps, and long reusable supports may receive an ordinary Vamana graph. Every internal node is built by an exact set operation: YFCC’s selected intersection plans use packed exact layouts, whereas the SIFT100M causal experiment makes graph layout an alternative. Block widths and graph/packed layouts enter Eq. (1) before selection. Common-subexpression reference counts ensure that a materialized node is charged once even when several formulas consume it.

SIFT100M isolates one DAG layer: its controlled metadata has 12 deterministic atoms, four each at approximately 0.5%, 5%, and 10% base selectivity. Its candidate universe is their 66 intersections. The counterfactual experiment prebuilds this finite universe so every arm replays identical artifacts; a deployed design loads and is charged only for selected graph and ID-map files. A representative calibration slice over the same predicate vocabulary estimates workload mass and plan latency; query vectors are disjoint from the held-out slice. Evaluation spans the complete non-benchmark pair universe across three orders of support magnitude; YFCC separately tests online composition of unmaterialized conjunction identities. A coverage/CVaR-aware formulation first satisfies the declared tail-coverage target and then maximizes conservative mean saving. This experiment tests non-benchmark queries over the predeclared 12-label pair universe. The evaluation reports no-pair, all-pair, benefit/byte, and smallest-cost controls. The YFCC workload’s older frequency-capped pair store admits all 3,335 observed calibration pairs and serves as an ablation. For ranges, alternative fixed-block families compete in calibration; the selected 200 width-25 blocks build one $R=32$ graph per block, so every point is represented once.

The universal conjunction fallback is a base-derived candidate family. Let $T = 2^{16}$. It stores an N -bit bitmap exactly for labels with base support greater than T (only 53 labels and 66,250,000 bytes on YFCC10M). A pair is oriented by (support, label ID). If the secondary support exceeds T , its bitmap filters the primary posting; otherwise both postings are bounded and a streaming merge emits their exact intersection. Primary-local offsets use 16 bits only when support is at most T ; the high–high case uses 32-bit global IDs and cannot truncate offset 65,536. This construction covers every label pair and uses no query vocabulary. A separately budgeted 824-label development vocabulary remains an optional accelerator, but it is not needed for correctness or coverage. The complete-q30 exact

endpoint enables 128 calibration-selected hot atoms in addition to the required high-support family; overlap leaves 148 bitmaps and approximately 185 MB.

4.2 Online plan selection

For an equality atom, the executor uses its local graph when admitted and an exact posting scan otherwise; the LAION scan may add a base-norm screen whose outward-rounded lower bound follows Proposition 2, with full exact enumeration on any certificate failure. For $a \wedge b$, it first checks the frozen pair dictionary. A hit executes that pair’s declared exact or graph layout. On a miss it invokes the vocabulary-complete exact-intersection fallback above, then scans contiguous vectors when the primary atom is packed and otherwise prefetches only the exact surviving global vectors. When a selected local graph is recall-eligible, the optimizer may instead use its frozen beam. If a required asset is absent, execution falls back to exact intersection; it never treats an absent asset as complete. Every returned ID is predicate-checked and all surviving candidates are reranked by the original exact distance.

Table 3: Calibrated cost surface for support P ; m is the emitted exact intersection and B is a graph beam. Exact plans enumerate a complete predicate support before distance ranking.

Plan	Charged state	Dominant query work	Contract
posting merge	sorted IDs	$ P_a + P_b + md$	exact
bitmap join	$N/8$ -byte bitmap	$ P_{pri} + md$	exact
packed scan	IDs + $ P d$ scalars	$ P d$ contiguous	exact
norm-screened scan	one norm/row	base bounded candidates + exact distance	exact
local graph	ID map + edges	$R P Bd$ + checked merge	empirical
block cover	one partition, RN edge slots	sum of covered blocks	exact (scan); empirical (beam)

Table 3 predicts crossovers rather than a uniformly best layout: merge pays when both postings are short, a bitmap removes the secondary-posting scan, packed vectors win while complete support enumeration remains cache-friendly, and a recall-eligible graph caps distance work on long supports. Calibration fits the omitted machine constants. Section 6.4 isolates the causal value of admitting one exact pair fragment and holds the logical range supports fixed and changes only their retained versus hybrid layout. The two interventions therefore measure support admission and layout selection, respectively.

The final YFCC design illustrates why the plan, rather than any one ANN structure, is the system contribution. Single-tag queries divide between selected Vamana local graphs and exact scans; every two-atom query uses an exact posting–bitmap or posting–posting join followed by packed/gather scan. Conjunctions use only base-derived posting and bitmap state. Thus all conjunctions are exact and every ranked-recall loss in the mixed workload comes from selected single-tag graph plans.

4.3 Ordered composition

A range decomposes into complete interior blocks and at most two partial boundaries. The executor searches each overlapping unit, rejects out-of-range boundary candidates, and merges the local pools by exact distance. An equality-guarded clause either

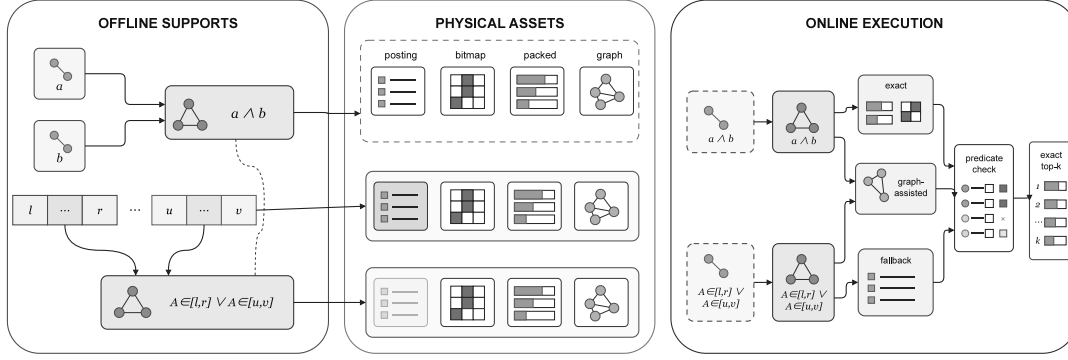


Figure 1: The Boole-ANN design object. Atoms and endpoint-independent blocks form one exact support DAG; internal supports can be shared by formulas and assigned alternative layouts. The categorical conjunction $a \wedge b$ and the separate two-clause range DNF $A \in [l, r] \vee A' \in [u, v]$ are examples from the implemented grammar that reuse the displayed nodes. Calibration selects fragment–layout–plan triples under storage, build, serving-memory, and calibrated tail-coverage goals. The frozen manifest enables exact or recall-eligible plans, while every route ends in the original predicate check and exact-distance merge.

intersects this cover with the guard support or slices its selected composite owner $C(t, A)$. A union of two clauses, over the same or different numeric attributes, executes the implemented two-clause range DNF; a one-dimensional negated interval is the union of its two complements. The index is independent of endpoints, and each attribute’s block family has exactly N graph memberships. Searching only a beam from each block makes recall empirical; enumerating the same block cover gives the exact tier. Approximate DNF eligibility is measured after the entire union, deduplication, predicate check, and global rerank; it is not inherited from per-clause recall.

4.4 Cost surface and fallback behavior

The materialization costs are explicit. An atom or pair graph on support P stores $R|P|$ directed edge slots plus its ID map; a packed exact pair stores $|P|$ IDs and $|P|d$ scalar coordinates; a dense membership bitmap costs $N/8$ bytes independent of tag frequency; and the complete fixed-block family stores RN edge slots because its supports partition the base. These quantities, not logical predicate count, are the capacities in Eq. (1).

For an uncached conjunction with sorted postings P_a and P_b , the exact fallback is a merge semi-join in $O(|P_a| + |P_b|)$ integer reads and computes distances only for intersection members. If the secondary bitmap is present, it instead scans the chosen primary in $O(|P_{\text{pri}}|)$ membership probes. The exact top- k heap adds $O(|P_a \cap P_b| \log(k+1))$ work. Approximate unit search replaces enumeration with a bounded beam but retains the complete ID map and final predicate check. A missing or corrupt unit activates the slower declared complete fallback.

4.5 Manifest validation and fallback semantics

The designer emits a manifest rather than executable source. Each row names a logical support, layout files, exact byte and edge charges, construction parameters, enabled operators, calibrated latency coefficients, recall eligibility, and a fallback plan. Shared

assets appear once and are reference-counted by plans. Startup validates row counts, local/global ID ranges, graph degree, block disjointness, and required files before accepting queries. Optional-row validation failure disables its dependent plans; mandatory-posting validation failure aborts startup.

For each request the predicate compiler performs four steps: (1) match the request against the supported grammar defined in Section 1; (2) enumerate only manifest-authorized plan templates; (3) discard plans that lack frozen whole-plan calibration eligibility for the requested recall target or a required asset; and (4) choose the minimum predicted cost. The API accepts precisely the grammar defined in Section 1. Execution returns global IDs, removes duplicates across branches, evaluates the original predicate, computes exact distances, and selects top- k by exact distance within the surviving candidate set, using ID as a secondary key when the backend provides stable tie ordering. If no specialized plan survives, the exact posting/scan fallback remains, converting a missing specialized path into additional latency while providing the mandatory tie-aware-exact tier.

Concurrency and accounting. One serving instance owns the index and eight worker-local scratch regions; queries are distributed dynamically, while each individual query executes on one worker. Consequently batch QPS measures eight-query offered concurrency, not an eight-thread acceleration of one competitor query. Competitors that expose only a single-query entry point are wrapped by eight independent calls over the same read-only index; their core search code and parameters are unchanged. Planner time, result conversion, deduplication, predicate checks, and exact reranking are inside the query interval. Dataset loading, index construction, and the post-run correctness audit are reported separately.

4.6 Implementation

One advisor emits the fragment–layout–plan manifest consumed by three execution backends: the categorical graph/posting engine (BCI), the contiguous Predicate-Cube configuration (X3), and the ordered-block engine for ranges and two-clause DNF. All three

use the same plan authorization, global-ID predicate check, exact-distance merge, fallback, and resource-accounting contract. They are distinct backends: the evaluation uses BCI to isolate support admission, X3 and PREFIX384 to isolate layout and exact execution, and the ordered engine to isolate endpoint composition under that shared contract.

The equality/conjunction engine uses $R=64$ Vamana local graphs, sorted 32-bit postings, dense 64-bit-word bitmaps, and contiguous byte-vector packs. The selected YFCC exact plan orients each conjunction by (support, tag ID). Primary supports at most 2^{16} emit checked 16-bit local offsets; larger supports use 32-bit global IDs. A boundary self-test rejects support 65,537 from the 16-bit path and verifies ID 65,536 through the global fallback. Eight workers own reusable, first-touched scratch buffers; allocation, build, and correctness audit are outside the measured batch, but every byte remains in peak RSS.

The range engine uses $R=32$ Vamana graphs over the selected disjoint blocks. All competitors retain their released core algorithms; adapters only translate the common vectors, predicates, thread count, and result IDs. Every system is run on identical query IDs and exact ground truth. Boole-ANN’s planning time is included in query latency; design/build time is reported separately.

For LAION, U24 stores each sorted global ID in three bytes. DELTA16 stores the first ID as a 32-bit integer followed by unsigned 16-bit gaps, and is eligible only when every gap is at most 65,535. Construction round-trips either encoding against the exact support and rejects an invalid or overflowing asset. A DIRECT asset stores one complete support, whereas FRAG composes endpoint-independent U24 blocks and checks the original predicate before scoring.

Complete sorted DNF supports can instead use an Elias–Fano owner. Each predicate indexes a 24-byte directory over a shared monotone payload; an eight-lane streaming decoder reconstructs global IDs and feeds the exact float32 top- k scorer without materializing an int32 support. The native audit checks the directory, decoded and scored row counts, ordering, and query self removal. A missing owner or any failed check routes to the immutable tie-aware-exact leaf path.

For wide exact supports, PREFIX384 stores 384 lossless half-precision coordinates per support row; all LAION base coordinates round-trip bit exactly. The coordinate order is fixed from base-wide second moments, without query or timing outcomes. Execution fully scores 32 prefix-nearest seeds, prunes a row only when a conservative prefix lower bound exceeds the current top- k threshold, and evaluates every survivor with the original 512-dimensional float32 scorer; its single-pass AVX2 loop widens eight stored coordinates per instruction.

Q8 maps each coordinate to one of 256 base-trained intervals and stores one byte. Interval separation gives a conservative squared-distance bound (tail codes contribute zero); only rows above the exact top- k cutoff are pruned, and float32 scoring verifies every survivor as in Proposition 2.

An endpoint-independent composite owner $C(t, A)$ stores every valid row in token t ’s exact posting in $(A(a_i), i)$ order, together with its selected PREFIX384 or Q8 payload. Binary search exposes a contiguous slice for $t \wedge A \in [l, r]$ without building endpoint-specific state. For a two-clause DNF, one union operator deduplicates the

two slices, chooses 32 prefix-nearest seeds globally, and applies one shared exact top- k threshold before rescoring every survivor. Fewer than 32 unique rows or any numerical anomaly activates complete float32 enumeration of the deduplicated union.

5 PHYSICAL-DESIGN GUARANTEES AND SCOPE

The deployed guarantees establish support correctness and exact-tier ranking; approximate graph plans use calibrated recall measured in §6. Build-time checks establish a bijection between every materialized unit’s local IDs and its declared posting. Consequently, atom and pair search cannot return a predicate-invalid ID; an exact scan over a complete atom or pair is exact filtered top- k . Posting merge and bitmap semi-join enumerate exactly the intersection they declare, so the vocabulary-complete exact-intersection path is exact regardless of the query-pair vocabulary.

PROPOSITION 3 (ENDPOINT-INDEPENDENT BLOCK COVER). *Let A be defined on every record, and let $I_1, \dots, I_B$ be a disjoint partition of an ordered domain containing every realized $A(a_i)$. Store $P_j = \{i : A(a_i) \in I_j\}$. Then $\sum_j |P_j| = N$. Any closed interval $[l, r]$ is the union of zero or more complete interior blocks and candidates from at most two boundary blocks; checking $l \leq A(a_i) \leq r$ on those candidates yields exactly its support. Complement relative to the base universe and finite union preserve exactness.*

PROOF. Every base value belongs to exactly one partition interval, establishing the membership sum. Every block strictly between the blocks containing l and r lies inside the query; only the two endpoint blocks can overlap without being contained. Exact endpoint checks remove their out-of-range IDs. Set-theoretic union, complement relative to $[N]$, deduplication, and the final predicate check therefore produce the exact requested support. $\square$

PROPOSITION 4 (CHECKED-PREFIX EXACTNESS). *Let $u = 2^{-24}$ and let $\hat{P}$ be the retained float32 sum of 384 nonnegative squared-difference terms selected from the 512 terms of a full retained score $\hat{T}$. Under round-to-nearest with contraction, FTZ, and DAZ disabled, $(1 - 2^{-14})\hat{P} \leq \hat{T}$. Consequently, strict pruning against that lower bound preserves every valid distance-exact top- k and, with reference-stable tie breaking, preserves T_k .*

PROOF. Nonnegativity gives $P \leq T$. The prefix eight-lane and full four-lane accumulators receive 48 and 128 rounded terms per lane. Counting 51 and 129 rounded additions gives $\hat{P} \leq (1 + u)^{51}P$ and $\hat{T} \geq (1 - u)^{129}T$. Since $1 - 2^{-14} \leq (1 - u)^{129}/(1 + u)^{51}$, the stated inequality follows; binary64 multiplication by this dyadic factor is exact (at most 38 significant bits). Equality is never pruned, and every retained candidate is rescored by the unchanged full scorer before ID-tie ordering. Startup rejects nonfinite inputs or an incompatible floating-point environment. $\square$

The same bound applies row-wise to two composite-owner slices. A single heap over their ID-deduplicated union supplies the shared threshold; hence strict pruning cannot remove any row below or at the global cutoff, and the complete-enumeration fallback covers every exceptional path.

Exact enumeration of every block in the cover is therefore exact filtered top- k ; deployed graph traversal over the same cover preserves validity but has the empirical recall reported in §6.2. The proof requires a complete partition family, which Eq. (2) encodes by selecting all blocks of one candidate family together.

The materialization budget is explicit: an atom or pair of support m costs Rm graph edges plus its ID map, while a fixed-block family costs RN edges. An absent requested fragment routes to the exact posting/scan fallback. The deterministic guarantees therefore cover predicate validity and exact-tier ranking for the supported fragment vocabulary. Approximate graph plans use empirically calibrated recall, and workload optimization ranges over the candidate fragments, layouts, and operators in Eq. (1).

PROPOSITION 5 (FAIL-CLOSED ASSET REMOVAL). *For every predicate in the supported grammar, removing an optional asset from a valid manifest cannot make an output predicate-invalid or eliminate the exact tier: every dependent plan is disabled and the mandatory exact fallback remains. A replacement exact plan is distance-exact (and stable exact when it uses the reference tie order); an approximate replacement retains its frozen group-level recall eligibility; and exact fallback has tie-aware recall one.*

PROOF. Plan authorization requires every asset in $R(p)$. Removing one therefore prevents partial execution of p . For the supported grammar, the fallback has $R(p) = \emptyset$, enumerates a support superset containing X_ϕ , and applies the original predicate and exact distance ranking, so Proposition 1 applies. Every remaining approximate plan has already passed the unchanged whole-plan calibration guard of Eq. (1). $\square$

6 EVALUATION

We organize the evaluation around three questions. **Q1** asks whether Boole-ANN outperforms evaluated released baselines at matched endpoints across equality, conjunction, range, and two-clause DNF workloads. **Q2** asks whether the advisor chooses effective designs under a byte budget and a calibrated fast-plan-coverage target, including mixed serving. **Q3** asks how much fragment materialization, physical layout, and plan selection each contribute. Sections 6.2–6.4 answer these questions in that order after a common setup. Figure 2 links them: predicate breadth tests the shared object, the 100M frontier tests scale, and controls attribute the gain to fragment, layout, and plan decisions.

6.1 Setup

Platform and measurement. All experiments run on one AMD EPYC 7642 server with 96 cores, 251 GB RAM, and no swap. Unless noted otherwise, search uses eight workers over one resident index, $k = 10$, and squared Euclidean distance, which preserves Euclidean rank order. QPS is completed queries divided by batch wall time; we also report request p50/p95/p99 latency and peak RSS where applicable. The SIFT and Parlay-IVF graphs use degree 64, while ordered range blocks use degree 32. Each comparison states its process, warmup, pairing, and inference unit; intervals use fresh processes or paired process blocks, never query rows or repeated passes, as independent samples. MB/GB denote decimal

units and MiB/GiB binary units. A query identity is one vector-predicate pair; an execution is one timed invocation, a pass repeats a fixed batch, and a paired block groups independently spawned arm processes. We reserve *fresh process* for a newly spawned OS process; *query-row-heldout* means the vectors never influence design or tuning, *predicate-class-heldout* means the formula identities do not, and *unseen* means absent from the design pool named by that experiment.

Data and workloads. Table 4 covers natural metadata and controlled predicates. SIFT100M supplies an exhaustive controlled 66-pair candidate universe and 58 non-benchmark pair classes whose supports span 2,430–1,000,928 records; SIFT10M uses published predicates and held-out numeric endpoints. YFCC10M and LAION1M supply real tags and caption tokens.

Table 4: Dataset and held-out protocol summary. Each experiment below fixes its detailed parameter grid and active cap before the listed test.

Dataset	N/dim	predicates	held-out contract
SIFT100K	$10^5/128$	eq/conj/range/DNF	disjoint rows and formulas; 24-MiB optional cap
SIFT10M	$10^7/128$	range/DNF	unseen endpoints; recall target .99
SIFT100M	$10^8/128$	conjunction	58 non-benchmark pairs crossed with post-freeze query blocks
YFCC10M	$10^7/192$	real tags	q10 and post-freeze q80/q90 tests
LAION1M	$10^6/512$	caption/range	predecessor-disjoint formulas and fresh streams; stated byte caps

Baselines and controls. Equality and conjunction experiments compare with Parlay-IVF and released SIEVE; range experiments compare with released WoW and SeRF, and the LAION mixed endpoint compares all four families with released SIEVE. The global label-aware lane is represented by ACORN. The artifact records the common applicability, resource, and correctness checks for the additional released UNG, iRangeGraph, Filtered-DiskANN, and JAG configurations. Only protocol-matched, recall-qualified, process-replicated configurations receive ratios; the artifact retains applicability and correctness outcomes for the rest. Competitors receive identical vectors, predicates, exact ground truth, offered concurrency, and warm-state policy. Adapters preserve released search algorithms and only translate inputs, invoke independent queries, and convert returned IDs. The materialization control disables the pair fragment while holding atom state, degree, beam, and router fixed. For two-clause DNF, both systems issue two native interval searches, check the original clauses, and merge by exact distance; the WoW adapter invokes the same released kernel for both clauses.

Splits, correctness, and resources. Base data determine candidate supports and byte costs; calibration selects designs, development fixes budgets and parameter grids, and held-out queries alter neither. All runs use exhaustive tie-aware ground truth; strict serialized recall is reported at cutoff ties, and returned IDs are validated. YFCC distances are recomputed with integer squared L2, an independent CPU implementation checks sampled supports and top- k answers, and a pre-timing control reproduces expected graph degree and baseline recall. We distinguish the optional-design budget from total persistent state and whole-process RSS/PSS, which include mandatory vectors, postings, allocators, descriptors, and

scratch. Each comparison reports its applicable cap together with total persistent and process-resident measurements. The advisor studies activate the stated optional-byte cap; build time and whole-process memory are measured and checked resource outcomes.

6.2 End-to-end performance across predicate families

Experiment 1: Natural tags against Parlay-IVF and ACORN.

We first vary the recall requirement on YFCC10M while holding queries, graph degree, worker count, exact ground truth, and warm-state policy fixed. YFCC follows BigANN’s public single- and two-tag protocol: qids [60K, 70K] calibrate the pair vocabulary, [0, 10K] choose the operating grid, and untouched [10K, 20K] queries form the primary frontier test; q80/q90 below is a second held-out endpoint after policy freeze. We abbreviate each 10K-query slice by its starting offset (q10, q30, q80, or q90). Both systems serve page-resident batches, with construction, load, and first touch outside the measured batch.

The exact online-composition design contains 2,106 frozen primary packs occupying 4.642 GB and 148 base-derived bitmaps occupying approximately 185 MB; pair intersections are computed online. The selected high-recall and exact endpoints are both positive against the strongest matched Parlay-IVF configuration.

The high-recall endpoint evaluates the least costly eligible policy, selected on development data before freeze. We then pair q80 vectors with a bijective, fixed-point-free permutation of q90 predicates and compute exhaustive ground truth. Across five opposite-order process pairs, Boole-ANN sustains 17.78–17.89k QPS versus 14.90–14.95k for Parlay-IVF R64, a 1.195 $\times$ geometric mean with 95% interval [1.191, 1.198]. Tie-aware recall is 0.99978 versus 0.99965 (strict 0.99977 versus 0.99964); all 18 passes per process return identical answers with zero predicate, duplicate-ID, or invalid-ID errors. The gain therefore comes from the least costly recall-eligible plan rather than uniformly greater graph effort.

The secondary development endpoint covers all 3,825 public-q30 two-tag queries. Three process-level ABBA campaigns hold IDs, CPUs, and worker budgets fixed without materializing query-pair rows; both systems return identical top-10 outputs at recall one. Boole-ANN/PIVF sustain 28,881/13,700 QPS, a 2.108 $\times$ ratio with interval [1.878, 2.366]; the q80/q90 experiment above supplies the held-out inference.

At this matched approximate endpoint, Boole-ANN and Parlay-IVF occupy 12.038/12.040 GB persistently and average 24.312/24.381 million reported kB of peak serving RSS. For exact composition, the corresponding totals are 4.827/12.040 GB and 7.56/24.4 million reported kB.

We also compare a mixed graph/exact policy with ACORN on 64 fresh identities: 37/27 use one/two atoms and, independently, 11/53 use graph/exact-prefilter execution. Disjoint calibration freezes $ef=2048$, the first tested value reaching .995 recall. Five independent paired units each aggregate four fresh-process blocks in alternating order on eight CPUs. Both arms time resident query execution after construction, loading, and support preparation. Boole-ANN obtains 5.65–6.01k QPS versus 110.6–112.8 for ACORN, a 52.486 $\times$ geometric mean with 95% interval [50.582, 54.463]. Both systems attain recall one in every row/width stratum with zero errors;

peak RSS is 23.284/12.336 million KiB and persistent index size is 12.038/11.826 GB (Boole-ANN/ACORN).

Forced plans on the same held-out q10 workload explain the portfolio result: exact scan gives 395 QPS at recall one, forced graph and cluster routes give 11,071/9,880 QPS at .91447/.92662 recall, and the frozen portfolio gives 12,607 QPS at .99947 recall. Sparse real-tag pair reuse thus favors base-derived atoms and online intersection; Section 6.4 tests the complementary high-reuse regime.

Experiment 2: SIEVE on controlled and natural workloads.

We compare with the closest workload-aware collection design at two scales. The first test holds a 400-query SIFT100K workload, a 24-MiB cap, and one selected Predicate-Cube package (X3) fixed. Queries are balanced across equality, conjunction, range, and DNF; query rows and conjunction/DNF identities are disjoint from design and earlier evaluation workloads. Five paired single-thread blocks alternate system order and rotate family order. Both systems consume identical uint8 vectors, SIEVE uses its released PartitionedHNSW<int, uint8_t> core, and design freezes $ef=100$.

X3 orders one vector payload by (bin, A, B, id), initializes τ with 256 full distances, and applies the safe prefix bound $d(q, x) \geq d_{64}^{(0)}(q, x)$ to 128-row microbatches. Complete cube support is the exact fallback. Including a 4-KiB transient, X3 charges 16.34 MiB to the optional-design cap; shared base state is reported separately. Figure 2a summarizes the formal per-family and mixed speedups against released SIEVE.

On SIFT100K, X3 reaches a 1.574 $\times$ mixed-workload geometric mean and all 20 family/block ratios exceed one, with pooled recall 1.000/.9995 (.998 for SIEVE ranges). A separate conjunction frontier is positive at all 11 $ef=100$ –200 points (1.557 $\times$; recall 1.000/.998909). There, Boole-ANN/SIEVE persistent state is 35.97/28.15 MB and preloaded RSS is 43.85/101.28 MiB.

The LAION1M q800 test is frozen after the final system (200 requests per family). Equality enumerates the finite 200-atom vocabulary with unseen query rows; conjunction, range, and DNF formulas are absent from both preceding pools. Boole-ANN serves equality, range, and DNF by guaranteed-exact support enumeration, and conjunction by a frozen 100K-vector-budget SIEVE collection; the released-SIEVE comparator contains all 42 candidate partitions. On the preceding development pool, the prespecified $ef \in \{200, 400, 800, 1600\}$ grid selects 800 as the first setting with .9995 recall in every family; 1600 provides no further throughput. Eight order-balanced process pairs yield 3.787 $\times$ QPS (3.676–3.837; two-sided exact sign-flip $p = 1/128$), and all 12,800 system executions match the exact top-10 IDs and float32 distances. SIEVE uses 165.1/536.1 MB persistent/runtime state versus Boole-ANN’s charged 245.8/554.8 MB, including an unqueried 86.0-MB local-equality graph.

Experiment 3: WoW and SeRF on range and DNF. At SIFT10M, two-clause DNF reuses 200 selected width-25 blocks and exactly N graph memberships. We test 1,000 held-out intervals of widths 26, 251, and 501 against WoW at frozen beam/ ef 160/1700, plus a fixed DNF workload. A distinct seed generates the DNF endpoints after the calibration gate and freezes them before ground-truth construction or search. *Unseen* denotes interval endpoints: block coverage is exact, while graph ranking is recall-qualified

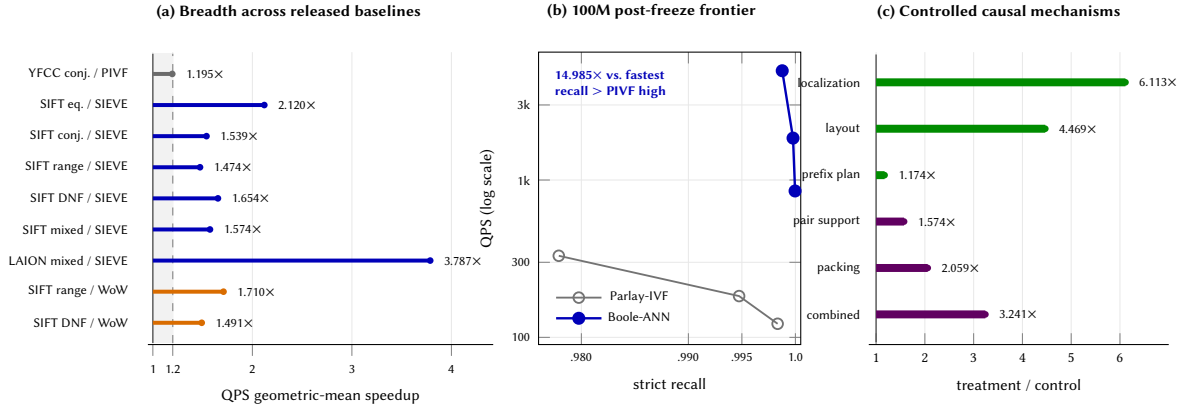


Figure 2: The experimental loop: breadth, scale, and causality. (a) Formal QPS geometric means against the released baseline named per row; the shaded band ends at the 1.20× practical margin. (b) On the fixed SIFT100M grid, Boole-ANN low lies above the displayed Parlay-IVF frontier in observed strict recall and QPS. (c) Controlled interventions isolate support, layout, and plan effects (green: 100K; purple: 100M). The text reports uncertainty, resources, and correctness audits.

end to end. Figure 2a shows both WoW comparisons above the prespecified 1.20× practical margin.

Intervals and p -values use one-sided log-ratio Student- t inference against the frozen 1.20× margin over eight range or five DNF paired-process blocks; the range bound is family-wise adjusted over its frozen frontier. Across eight order-balanced range pairs, the geometric-mean gain is 1.710×; the family-wise one-sided 95% lower bound is 1.682× against the prespecified 1.20× margin ($p < .001$). Strict recall is 0.9965 versus WoW’s 0.9944 (tie-aware 0.9966/0.9945), and complete-process peak PSS is 6.01/14.90 GiB with zero swap.

DNF composes two range-clause candidate pools through predicate checking, deduplication, and global exact reranking. In five fresh processes on the fixed workload, all returned IDs satisfy the original predicate. The geometric-mean gain is 1.491× and the one-sided 95% lower bound is 1.440×, above the 1.20× margin ($p < .001$); recall is 0.9938 versus 0.9945, above the declared 0.99 target for both systems.

The released SeRF path serves as a recall-qualification control. Across $ef = 8,192/16,384/32,768$, its measured recall is .9704/.9712/.9715; none reaches the prespecified .99 target, so Table results report no speed ratio against SeRF. All returned IDs are valid and unique. The same 1.292 GB Boole-ANN block index serves range and DNF, versus 14.080 GB for WoW; matched builds take 553/7,491 s. These results extend the end-to-end comparison from categorical predicates to unseen ranges and composed DNF.

6.3 Budgeted design and mixed serving

Experiment 4: Fragment admission under a fixed budget. We isolate the advisor from the complete serving path by fixing the search kernel and varying only which exact pair supports fit a 2.5 GiB budget. The workload contains 58 active SIFT100M pair classes and 100 logical queries per class in each of five processes. The first 20 queries per class estimate fragment value; the remaining 80 produce $58 \times 80 \times 5 = 23,200$ test replay observations. Within this retrospective calibration/test replay, selection uses calibration

latency and exact fragment bytes, never test latency. The coverage/CVaR rule enumerates the ten budget-feasible 56-support sets and uniquely selects the set omitting tags 20910 and 20911. The benefit/byte control instead reapplies the separately frozen eight-pair calibration rule; it admits 61 of 66 catalogue pairs and covers 54 of the 58 evaluated classes. This first control holds out query rows within fixed pair identities; the transfer study below instead holds out 58 pair identities from the eight used for calibration.

The cheapest 56 active supports consume 2,620,392,060 bytes and fit, whereas the cheapest 57 consume 2,845,162,220 bytes and do not. Thus $56/58 = 96.55\%$ is the maximum uniform coverage. The coverage/CVaR rule first attains that coverage and then minimizes calibration CVaR₉₅ and mean latency.

Table 5: At 2.5 GiB, counterfactual query-row-heldout replay of separately measured pair-on/off traces shows that coverage-first admission removes the p95 fallback tail. The final column is $1000/\bar{L}$, not batch QPS.

Design	active	charged GiB	mean ms	p95 ms	$1000/\bar{L}$
no pair asset	0/58	0	83.738	98.758	11.94
benefit/byte	54/58	2.391	9.136	85.695	109.46
max-coverage/cheapest	56/58	2.440	6.245	7.049	160.13
coverage/CVaR replay	56/58	2.441	6.217	7.041	160.85
all active (over budget)	58/58	2.859	3.326	6.564	300.66

Table 5 shows that the coverage/CVaR design improves the reciprocal-mean-latency rate by 1.470× and reduces p95 by 12.17× over benefit/byte; process-level p95 is 6.57–7.22 ms. Within the maximum-coverage set, CVaR tie-breaking further reduces both mean and p95 latency.

Table 6 separately tests predicate-class transfer over eight byte budgets. Policies are frozen from eight calibration pairs and replay all 100 queries in each of 58 other pair classes across five processes, giving $58 \times 100 \times 5 = 29,000$ observations per policy. For selected set S , predicate-class-heldout utility is the equally weighted positive

mean-latency saving

$$U(S) = \frac{1}{58} \sum_{i \in S \cap H} \max(0, \bar{L}_i^{\text{off}} - \bar{L}_i^{\text{on}}),$$

where H is the 58-class held-out set and fragments selected outside H consume budget without adding held-out saving. U_{oracle} maximizes this post-test quantity under the same byte budget and is used only for evaluation.

Table 6: Across eight budgets, advisor utility remains within 5.59% of the predicate-class-heldout oracle.

budget	S	used MiB	$U(S)$ ms	U_{oracle} ms	regret
0	0	0.0	0.00	0.00	0.00%
128 MiB	27	128.0	32.12	33.58	4.36%
256 MiB	38	236.5	47.18	47.18	0.00%
512 MiB	43	495.5	53.21	54.75	2.82%
1 GiB	48	1023.7	60.52	62.18	2.68%
2 GiB	58	2024.4	72.13	73.78	2.24%
3 GiB	63	3040.9	76.48	81.00	5.59%
all	66	3520.1	81.00	81.00	0.00%

At 1 and 2 GiB, predicate-class-heldout mean latency is 23.89 and 12.28 ms, versus 25.37 and 13.76 ms for the strongest frozen benefit-per-byte control. All eight HiGHS solutions reproduce the frozen selections exactly in 40.6 ms total (10.2 ms maximum per budget), with certified relative gap at most 1.6×10^{-16} . Real selected-manifest loads at 1–3 GiB stay within 3.83% of replay means, retain recall at least 0.99981, load in 10.19–12.01 s, and peak at 65.64–67.96 GiB RSS.

Query-free scale controls preserve the resource contract: the 100K degree-one case reduces 600K variables to 484, reaches zero gap in 4.842 s, and peaks at 390.90 MiB; the connected 10K case solves 70K variables in 37.64 s at 287.14 MiB with a 3.654% relative optimality gap.

These results expose the coverage transition. Coverage-first admission moves the required workload mass off the 80–100 ms fallback; within that set, CVaR selection is 0.45% faster than choosing the cheapest supports. On real-tag YFCC, the selected design uses base-derived bitmaps and online intersection, matching the measured sparse pair reuse.

Experiment 5: Mixed, budgeted, and checked exact serving. Complementing the released-baseline comparison in Figure 2a, this experiment contains one integrated system study and three contrasts over two separately frozen selector populations. The integrated study executes 800 unique LAION request identities (200 per family) for four steady cycles under one 13.476-MB manifest and a 16.244-MB cap. It balances equality, conjunction, range, and cross-attribute two-clause range DNF in every process. An outcome-independent query-by-example policy takes each query from one base row, which both ground truth and search remove. Four routing arms retain identical state and scratch while enabling selected, direct-only, fragment-only, or leaf-only dispatch. The selector contrasts use two frozen populations under the same 7,227,208-byte cap.

In Table 7A, one common-manifest mixed request stream rather than an aggregate of family-specific measurements, eight fresh Latin-order blocks return bit-identical IDs and float32 distances at strict and tie-aware recall 1.000 in every arm. The 10,730-object manifest holds 772 recurring conjunction supports and

Table 7: LAION evidence. Panel A is one common-manifest four-family end-to-end campaign; Panels B–D are same-cap selector contrasts over two separately frozen populations. Absolute rows report QPS and milliseconds; ratio direction is explicit in each contrast.

<i>A: mixed end-to-end, 16.244-MB cap</i>					
endpoint	strict recall	QPS	p95 ms	p99 ms	
common manifest (200/family ×4)	1.000	1,680	10.58	17.36	
leaf-only (200/family ×4)	1.000	1,079	22.65	43.58	
selected / leaf-only	—	1.557×	0.467×	0.398×	
<i>B: full-vocabulary selector, 7,227,208-byte cap</i>					
contrast	strict recall	QPS ratio	p95 ratio	p99 ratio	
joint owner / two-stage (1,957 × 6)	1.000/9995	3.696×	0.331×	0.373×	
<i>C–D: selector contrasts, 7,227,208-byte cap</i>					
contrast (ratio direction)	geometric mean	block range	exact p		
service mean latency (staged/joint; 206 DNF)	1.362×	1.295–1.484	1/64		
advisor wall time (reoptimized staged/joint)	63.786×	63.498–64.169	1/32		

9,958 fixed blocks, occupying 13,475,748 of 16,244,160 bytes. It routes each block’s 800 equality invocations through direct support, 1,600 conjunction/DNF invocations through materialized fragments, and 800 range invocations through the manifest-authorized complete fallback. Relative to same-resident leaf-only execution, it improves complete-wall QPS by 1.557× while reducing p95/p99 to 0.467/0.398×. The direct, fragment, and interaction contrasts are positive in all eight blocks (one-sided exact sign-flip $p = 1/256$). Identical objects, descriptors, scratch, and request order isolate dispatch and execution across 32 child processes.

A separately frozen 16.244-MB configuration tests a second 3,200-request stream whose conjunction, range, and DNF formulas are all predecessor-disjoint while retaining their leaf routes. Activating the selected equality route for 582 requests raises complete-stream QPS by 1.178× and reduces p95/p99 to 0.621/0.485× across eight fresh process pairs; all 51,200 executions return bit-identical IDs and float32 distances. This control measures mixed-stream robustness while the non-equality structured predicate vocabulary changes.

A disjoint route-choice audit evaluates every request with two executable alternatives. The frozen rule selects the lower-latency route for 597 of 600 requests; its sum-normalized regret to a per-request hindsight oracle is 0.063% on those requests and 0.050% over the complete four-family stream. Thus the mixed result reflects accurate held-out plan choice rather than only fast individual operators.

Table 7B uses a full-vocabulary joint treatment that replaces the initially admitted conjunction, range, and norm-screening assets with complete DNF supports in an Elias–Fano owner and assigns their streaming direct plan; two-stage retains its initially admitted fragments and composition path. Six ABBA blocks cover all 1,957 fixed DNF predicates with a different, previously unexposed support-member query vector per block. Under the common 7,227,208-byte cap, the joint bundle improves complete-batch throughput by 3.696× (block range 3.571–3.824; one-sided exact sign-flip $p = 1/64$) and reduces p95/p99 to 0.331/0.373×; all four

support-cardinality strata improve by 2.917–4.909 $\times$. All 46,968 executions are tie-aware exact (serialized strict joint/two-stage recall 1.000/.999523, differing only at cutoff ties), with exact float32 distance bits and valid IDs. The arms use 6,671,700/6,445,215 optional bytes; the owner builds in 5.03 s at 3.01 GiB peak. For 64 unexposed DNF formulas, all 384 owner misses enter the exact leaf and match independently recomputed supports.

Table 7C uses the same 7,227,208-byte accounted design cap. A position-balanced schedule over six fresh-instance blocks compares the joint manifest with the strict two-stage control. Two-stage freezes conjunction and range-support fragments before DELTA16 encoding and cannot refill or reassign the released capacity; joint selection can reinvest it. The manifests execute identically outside their 206-route decision boundary; over that population, joint selection gives a 1.362 $\times$ mean-latency speedup over two-stage (range 1.295–1.484; exact one-sided sign-flip $p = 1/64$). All 1,236 distinct query instances return the exact top-10 ID set, with no invalid or duplicate ID; two-stage/joint occupy 6,445,215/7,085,135 bytes. This frozen-decomposition control isolates whether layout savings can alter fragment admission; the changed factor is reinvestment, not an execution kernel.

Table 7D evaluates a competitive staged advisor that retains Stage 1 admission but permits exact Stage 2 reoptimization, repeated capacity refill, and add/drop/swap/support-removal search to a certified fixed point. On one fixed, previously exposed development q800 candidate instance, five fresh, strictly sequential paired process blocks use the same complete candidate ledger, coefficients, 7,227,208-byte cap, and eight-core allocation. The staged advisor takes 12 refill rounds (11 accepted) and two fixed-point sweeps to recover exactly the joint optimizer’s 561 plans, objective, and byte count. From a candidate-ready ledger to durable manifest, byte-ledger, and plan-policy publication, staged/joint wall times have geometric means 651.827/10.219 s, a 63.786 $\times$ ratio (range 63.498–64.169 $\times$; one-sided 95% lower bound 63.513 $\times$; exact sign $p = 1/32$); the CPU-time ratio is 350.100 $\times$. Thus the service difference in the strict control measures irreversible staging, while the reoptimized control measures the computational cost of recovering from it.

6.4 Scale and causal mechanisms

Experiment 6: Post-freeze SIFT100M frontier. We cross 56 post-freeze query vectors with all 58 non-benchmark conjunction classes, yielding 3,248 queries. Each point in Table 8 uses five fresh eight-worker processes and six complete passes; the sixth pass is fixed prospectively. Both systems use degree 64, and Parlay-IVF executes through its unchanged official batch-filter interface. Boole-ANN low/mid/high use beam/pool 256/1,024, 640/2,560, and 1,536/6,144; Parlay-IVF uses target-points/tiny-cutoff/beam 0.4M/2M/700, 0.8M/3M/1,000, and 1.2M/3M/1,400.

Across the fixed grid, all nine cross-tier comparisons satisfy the frozen observed-recall criterion: the Boole-ANN tier’s strict-recall point estimate is at least that of the compared Parlay-IVF tier. Their family-wise QPS lower bounds exceed 1.20 $\times$; the smallest is 2.207 $\times$. At the observed endpoints, the low-effort Boole-ANN point lies above the complete Parlay-IVF frontier: its strict-recall point estimate exceeds the highest-recall Parlay-IVF point while its throughput is 14.985 $\times$ higher than the fastest Parlay-IVF point. Across

Table 8: Official Parlay-IVF frontier over all 58 non-benchmark SIFT100M classes and a post-freeze query block. Rows show same-effort Bonferroni–Welch intervals; the simultaneous correction covers all 3 $\times$ 3 cross-tier comparisons.

effort	strict recall (ours/PIVF)	QPS (ours/PIVF)	ratio [95% CI]
low	.99877/.97789	4,949/330	14.985 [13.823,16.245]
mid	.99978/.99474	1,847/183	10.106 [8.680,11.765]
high	.99997/.99834	852/122	7.006 [5.969,8.225]

180 complete passes, all 5,846,400 returned distances pass bitwise validation, with no invalid ID, duplicate, predicate failure, incomplete result, fallback, or swap. Persistent state is 3.691/14.396 GB for Boole-ANN/Parlay-IVF, with 0.150 GB of derived Boole-ANN state; maximum peak RSS is 53.24/68.80 GiB. QPS inference treats the five processes per cell as independent; recall is the descriptive micro-average over the complete fixed 3,248-query grid.

Experiment 7: Support, layout, and plan controls. At SIFT100K, we hold queries, state, kernels, and the 24-MiB cap fixed and disable one choice at a time. Figure 2c reports five rotated blocks per control; Section 6.2 gives the standalone conjunction frontier.

Removing support localization, contiguous layout, or range-prefix pruning reduces mixed throughput by 6.113/4.469/1.174 $\times$; all 30 processes remain exact. Counters show 14.713 $\times$ row amplification, 99.780% nonadjacent gathers, and 81.859% avoided suffix evaluations. Every arm stores 17,132,772 bytes; transient bounds of 4,096 or 408,192 bytes remain below the common cap, and mean peak RSS is 86,256–86,325 KiB. These fixed-state controls show how support, layout, and plan choices contribute to the measured same-kernel gap.

We repeat the support–layout chain at SIFT100M, crossing the same 58 non-benchmark conjunction classes with a separately post-freeze 56-query block (3,248 queries). Across five independently spawned, order-rotated blocks, exact pair support improves exact QPS by 1.574 $\times$ over atom gather; packing that support adds 2.059 $\times$, for a combined 3.241 $\times$. Its one-sided 95% lower bound is 3.152 $\times$. Pair-arm candidate work is identical, whereas atom gather visits 11.316 $\times$ more candidates. Across all 15 processes and 90 passes, all 2,923,200 top-10 ranks match exact IDs and float32 distance bits, with zero swap.

7 CONCLUSION

Boole-ANN makes compositional exact supports a physical-design unit and jointly selects layouts and authorized plans under resource and calibrated-coverage goals. Within the stated grammar, matched comparisons and controls connect gains to support reuse, layout, and plan choice. Checked routes guarantee predicate validity, complete posting/scan plans supply the tie-aware-exact tier, and graph routes remain recall-qualified. Structured filtered vector search therefore benefits from designing support and vector access together.

ACKNOWLEDGMENTS

This work was supported by the National Key R&D Program of China (Grant No. 2024YFC3308200).

REFERENCES

- [1] Sanjay Agrawal, Surajit Chaudhuri, and Vivek R. Narasayya. 2000. Automated Selection of Materialized Views and Indexes for SQL Databases. In *Proceedings of the 26th International Conference on Very Large Data Bases*. 496–505.
- [2] Anas Ait Aomar, Karima Echihabi, Marco Arnaboldi, Ioannis Alagiannis, Damien Hilloulin, and Manal Cherkaoui. 2025. RWalks: Random Walks as Attribute Diffusers for Filtered Vector Search. *Proc. ACM Manag. Data* 3, 3 (2025), Article 212. <https://doi.org/10.1145/3725349>
- [3] Abylay Amanbayev, Brian Tsan, Tri Dang, and Florin Rusu. 2026. Filtered Approximate Nearest Neighbor Search in Vector Databases: System Design and Performance Analysis. *arXiv preprint arXiv:2602.11443* (2026). <https://arxiv.org/abs/2602.11443>
- [4] Yuzheng Cai, Jiayang Shi, Yizhuo Chen, and Weiguo Zheng. 2024. Navigating Labels and Vectors: A Unified Approach to Filtered Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 2, 6 (2024). <https://doi.org/10.1145/3698822>
- [5] Surajit Chaudhuri and Vivek R. Narasayya. 1997. An Efficient, Cost-Driven Index Selection Tool for Microsoft SQL Server. In *Proceedings of the 23rd International Conference on Very Large Data Bases*. 146–155.
- [6] Joshua Engels, Benjamin Landrum, Shangdi Yu, Laxman Dhulipala, and Julian Shun. 2024. Approximate Nearest Neighbor Search with Window Filters. *CoRR* abs/2402.00943 (2024). <https://arxiv.org/abs/2402.00943>
- [7] Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, Yin Zhang, Neelam Mahapatro, Premkumar Srinivasan, Amit Singh, and Harsha Vardhan Simhadri. 2023. Filtered-DiskANN: Graph Algorithms for Approximate Nearest Neighbor Search with Filters. In *WWW*. DOI 10.1145/3543507.3583552.
- [8] Hao Guo, Jiwei Shu, and Youyou Lu. 2026. PipeANN-Filter: An Efficient Filtered Vector Search System on SSD. *arXiv preprint arXiv:2605.17992* (2026). <https://arxiv.org/abs/2605.17992>
- [9] Mengxu Jiang, Zhi Yang, Fangyuan Zhang, Guan hao Hou, Jieming Shi, Wenchao Zhou, Feifei Li, and Sibao Wang. 2025. DIGRA: A Dynamic Graph Indexing for Approximate Nearest Neighbor Search with Range Filter. *Proc. ACM Manag. Data* 3, 3 (2025), Article 148. <https://doi.org/10.1145/3725399>
- [10] Zhaozheng Li, Silu Huang, Wei Ding, Yongjoo Park, and Jianjun Chen. 2025. SIEVE: Effective Filtered Vector Search with Collection of Indexes. *Proc. VLDB Endow.* 18, 11 (2025), 4723–4736. <https://doi.org/10.14778/3749646.3749725> <https://github.com/BillyZhaozhengLi/SIEVE-vldb25>
- [11] Zhonggen Li, Haoran Yu, Zixuan Xu, Yifan Zhu, and Yunjun Gao. 2026. A GPU-Accelerated Framework for Multi-Attribute Range Filtered Approximate Nearest Neighbor Search. *arXiv preprint arXiv:2604.20121* (2026).
- [12] Yingfan Liu, Tong Wu, Jiadong Xie, Yang Zhao, Jeffrey Xu Yu, and Jiangtao Cui. 2026. Generalized Range Filtering Approximate Nearest Neighbor Search: Containment and Overlap. *arXiv preprint arXiv:2605.26474* (2026). <https://arxiv.org/abs/2605.26474>
- [13] Jiarui Luo, Miao Qiao, Chaoji Zuo, and Dong Deng. 2025. Tag-Filtered Approximate Nearest Neighbor Search. In *IEEE ICDE*. 3642–3654.
- [14] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. 2024. ACORN: Performant and Predicate-Agnostic Search Over Vector Embeddings and Structured Data. *Proc. ACM Manag. Data* (2024). <https://doi.org/10.1145/3654923>
- [15] Zhencan Peng, Miao Qiao, Wenchao Zhou, Feifei Li, and Dong Deng. 2025. Dynamic Range-Filtering Approximate Nearest Neighbor Search. *Proc. VLDB Endow.* 18, 10 (2025), 3256–3268. DOI 10.14778/3748191.3748193; <https://github.com/rutgers-db/DynamicSegmentGraph>.
- [16] Gaurav Sehgal and Semih Salihoglu. 2025. NaviX: A Native Vector Index Design for Graph DBMSs With Robust Predicate-Agnostic Search Performance. *Proc. VLDB Endow.* 18 (2025).
- [17] Harsha Vardhan Simhadri, Martin Aumuller, Amir Ingber, Matthijs Douze, George Williams, Magdalen Dobson Manohar, et al. 2025. Results of the Big ANN: NeurIPS’23 Competition. In *Advances in Neural Information Processing Systems 38 (Datasets and Benchmarks Track)*. <https://big-ann-benchmarks.com>.
- [18] Junjie Song, Yu Liu, Guoyu Hu, Zhongle Xie, Ming Yang, Beng Chin Ooi, and Ke Zhou. 2026. FAVOR: Efficient Filter-Agnostic Vector ANNS Based on Selectivity-Aware Exclusion Distances. *Proc. ACM Manag. Data* 4, 3 (2026), 1–25. <https://doi.org/10.1145/3802060>
- [19] Yitong Song, Bin Yao, Zhida Chen, Xin Yang, Jiong Xie, Feifei Li, and Mengshi Chen. 2025. Efficient Top-k Spatial-Range-Constrained Approximate Nearest Neighbor Search on Geo-Tagged High-Dimensional Vectors. *The VLDB Journal* (2025). <https://doi.org/10.1007/s00778-024-00894-5>
- [20] Ziqi Wang, Jingzhe Zhang, and Wei Hu. 2025. WoW: A Window-to-Window Incremental Index for Range-Filtering Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 3, 6 (2025), 1–27. <https://doi.org/10.1145/3769843>
- [21] Tianming Wu and Dixin Tang. 2025. PathFinder: Efficiently Supporting Conjunctions and Disjunctions for Filtered Approximate Nearest Neighbor Search. *arXiv preprint arXiv:2511.00995* (2025).
- [22] Jingyi Xi, Chenghao Mo, Benjamin Karsin, Artem Chirkin, Mingqin Li, and Minjia Zhang. 2025. VecFlow: A High-Performance Vector Data Management System for Filtered-Search on GPUs. *Proc. ACM Manag. Data* (2025). DOI 10.1145/3749189.
- [23] Haikue Xu, Guy Blelloch, Laxman Dhulipala, Lars Gottesbüren, Rajesh Jayaram, and Jakub Łącki. 2026. JAG: Joint Attribute Graphs for Filtered Nearest Neighbor Search. *arXiv:2602.10258*. Open-source: github.com/xuhaike/JAG.
- [24] Yuexuan Xu, Jianyang Gao, Yutong Gou, Cheng Long, and Christian S. Jensen. 2024. iRangeGraph: Improvising Range-dedicated Graphs for Range-filtering Nearest Neighbor Search. *Proc. ACM Manag. Data* 2, 6 (2024). <https://doi.org/10.1145/3698814>
- [25] Chunxiao Ye, Xiao Yan, and Eric Lo. 2025. Compass: General Filtered Search across Vector and Structured Data. *arXiv preprint arXiv:2510.27141* (2025). <https://arxiv.org/abs/2510.27141>
- [26] Xiaowei Ye, Rong-Hua Li, Guoren Wang, Kaiwen Xue, Daiyin Wang, and Xubin Li. 2026. MCI: A Maximal Clique Index for Efficient Arbitrary-Filtered Approximate Nearest Neighbor Search. *arXiv preprint arXiv:2604.22171* (2026). <https://arxiv.org/abs/2604.22171>
- [27] Yuanhang Yu, Dawei Cheng, Ying Zhang, Lu Qin, Wenjie Zhang, and Xuemin Lin. 2026. Efficient Approximate Nearest Neighbor Search under Multi-Attribute Range Filter. *arXiv preprint arXiv:2602.15488* (2026). <https://arxiv.org/abs/2602.15488>
- [28] Zhiqiu Zou, Ziqi Yin, Rong-Hua Li, Hongchao Qin, Qiangqiang Dai, and Guoren Wang. 2026. RNSG: A Range-Aware Graph Index for Efficient Range-Filtered Approximate Nearest Neighbor Search. *arXiv preprint arXiv:2603.12913* (2026). <https://arxiv.org/abs/2603.12913>
- [29] Chaoji Zuo, Miao Qiao, Wenchao Zhou, Feifei Li, and Dong Deng. 2024. SeRF: Segment Graph for Range-Filtering Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 2, 1 (2024), Article 69. <https://doi.org/10.1145/3639324>